ComponentGraph and ComputationGraph: what changed, what was fixed, and what is unfinished

Original comparison snapshot: 13 October 2026. Query-host consolidation update: 16 October 2026. Native fan-out, bootstrap and consumer update: 20 October 2026.

The original review compared the older ComponentGraph-based runtime at Core `261b8d40` with the ComputationGraph snapshot at Core `707d27c5` and Server `f15706d`. The current descriptions include the subsequent published changes through Core `54735c05` and Server `15dafa2`. Historical defect comparisons still use the pinned old revision. Later qualification is identified separately; it is not a fresh rerun of every historical suite. The current `ComponentGraphView` is a read-only inspection view, not the old execution system being compared; the `ComponentGraph` alias has been removed. Here **native** means a component using the newer processing/plugin interface, not merely code compiled for the local machine.

This is not a published-release announcement. Code, recorded tests and unverified claims are distinguished below. This is a source-and-evidence review, not a fresh rerun of every test suite. The previous claim that the entire execution-and-isolation work package was complete was too strong: the full deadline policy and graph-size publication requirement remain unresolved. The cleanup plan records current migration status; the review inventory retains the broader findings and earlier qualification checkpoints.

In plain English

ComputationGraph lets an application build a processing pipeline from separately managed steps: read data, validate or change it, run queries, save results and send them elsewhere. It can run steps without a query, keep selected work safe through restart, manage their resources, and run normal DrasiLib instances in separate processes when crash isolation is needed.

There are three different achievements:

- **More ways to build applications:** reusable processing steps, explicit delivery choices, saved native definitions and connected child instances.
- **Fixes to real problems:** several older input, recovery and shutdown paths could lose work, use stale state or report cleanup more confidently than the code justified.
- **Clearer limits:** receiving an input, saving it, processing it and finishing an external action are different events. A guarantee at one step is not a guarantee that the entire application finished exactly once.

The old runtime already had continuous queries, transactions and recovery. ComputationGraph did not invent those. Some work preserved existing behavior; some generalized it; some corrected bugs.

The latest work restores an important native requirement: one producer can feed many processing branches without duplicating that producer or pretending all branches have the same saved progress. It also lets one query initialize from multiple real providers. PostgreSQL row and whole-transaction groups,

mixed PG/HTTP initialization and Trading's actual query bundle now exercise those contracts. This completes that slice, not the entire connector/release migration.

Ordinary queries now run directly in the instance graph

16 October 2026 update. Earlier ComputationGraph builds put an ordinary DrasiLib query inside a `QueryInstance` host with its own small private graph. The actual query processor inside that graph was already `ContinuousQueryTransformer`. The host and private graph have now been removed: ordinary and explicitly assembled native queries use that same processor in the instance's main graph. The shared `drasi-core` query engine was not replaced.

Each query's input readers, timer helper and result outlet are also main-graph components. Starting or stopping an ordinary query operates on its own helpers without stopping a shared source or an unrelated query. A cleanup gap found during this consolidation is fixed: individual and bulk stop APIs still finish the helpers when the evaluator itself has already stopped. This was a ComputationGraph consolidation defect, not another old ComponentGraph defect.

Server retains ordinary query-creation APIs, with breaking-release cleanup of obsolete settings. Query declarations retain their settings, dependency waits and actual failures for inspection. Saved-state and producer identities now use the actual instance, graph and query scope. The old `lib-query/<hex query ID>` namespace is not translated or deleted. Reopening the same current-release scope preserves checkpoints, result rows and unconfirmed output.

The standard query factory now constructs `ContinuousQueryTransformer` directly with output-delivery tracking. The redundant query constructors on `TransactionTransformer` have been removed; direct callers use the query component and opt into delivery tracking. Queries are still built in. Making the processor an optional dynamically loaded query plugin remains a separate, unfinished step.

See the ordinary-query assembly, query factory, saved-state relocation regressions and whole-query stop regressions.

What the old runtime already did

The inspected ComponentGraph version already had:

- A shared component registry, dependency information and lifecycle status.
- Sources, continuous Cypher/GQL queries, reactions, middleware and initial snapshots.
- Independently running query workers, including use of multiple CPU workers.
- Shared sources with multiple query subscribers and query results with multiple reactions; fan-out itself was not invented by the new graph.
- Persistent indexes, saved source positions, current-result snapshots and saved outgoing results, usually called the **outbox**.
- Supported query transactions that saved index changes, source progress and outgoing results together.
- Replay, consumer recovery, credentials, provider integration, logs and metrics.

It also already signalled due timer work without removing it; actual timer processing/removal happened in the query transaction. That particular behavior must not be advertised as a newly fixed old-runtime defect.

Evidence is in the old registry, query manager and transaction staging, result/outbox state and timer source.

Existing features that had bugs or incomplete failure behavior

These are problems in the inspected old implementation, not claims that all old queries or transactions were broken. Each entry explains what changed and the limits of the evidence. Later ComputationGraph-only bugs are not relabelled as old ComponentGraph defects.

A storage read error could trigger deletion of valid recovery state

Old problem: if reading a saved query configuration hash failed, the old query manager could enter a “clear persistent state and start fresh” path. A read error is not evidence that the configuration changed or that saved work may safely be discarded.

What changed: current query recovery propagates that read error instead of using it as permission to reset. A deployment must fix the storage problem or take an explicit supported recovery action.

Evidence and limits: the old error branch and current recovery code show the change. This review did not locate a standalone current regression that injects this exact configuration-hash read error. Broader passing recovery tests are not proof of that particular trigger. The missing explicit evidence belongs in CW-39, not a claim that the code change is absent.

Timestamp sorting could skip an input that had never been processed

Old problem: the query queue sorted all available events by timestamp before source sequence. If a source's later event had an earlier timestamp, it could be processed first. The saved input position then advanced past an earlier event, and the replay filter could discard that earlier event as “already processed.” This could lose a different row's change, not merely reorder two equivalent updates.

What changed: supported queues preserve each producer's input order and compare the next available events from different producers. A backward clock cannot move the saved position past still-unprocessed input from that producer.

Evidence and limits: compare the old queue ordering and sequence filter with the current queue implementation and query input code/tests. The focused backward-timestamp regressions check that both source rows and reaction operations are processed rather than skipped. This fixes the already-queued input ordering problem. It does not promise to wait for every unseen earlier event from every source; bounded source-time reordering is a separate capability below.

Repeated inserts could count the same row more than once

Old problem: the shared evaluator handled an Insert as a new contribution without first applying the current replacement rules for an already present identity. Re-delivering an insert under a new input sequence could therefore produce incorrect aggregates even though the index ultimately held one row.

What changed: an insert for an existing identity replaces that row and adjusts its previous contribution. An identical insert is a no-op; a changed insert updates rather than double-counting it.

Evidence and limits: old Insert processing versus the current duplicate/upsert tests and persistent independent model. This is a shared evaluator fix used by native queries, not a new aggregation feature.

Old updates or deletes could undo newer data; deleted rows could reappear

Old problem: the evaluator used the incoming change without the current source-time checks. After a delete, it also lacked the current saved deletion history needed to reject an older insert after reconstruction.

What changed: strictly older mutations are reported and ignored while input progress still advances. Deletion history is saved in the supported query transaction, so a late old insert cannot resurrect a deleted row after restart. Equal-time differences follow admitted order.

Evidence and limits: old mutation processing, current evaluator tests and RocksDB reconstruction tests cover the stated rules, rollback and saved positions. This is not a global ordering guarantee or permission to expire deletion history arbitrarily.

A node and relationship could reuse one live identity without the current check

Old problem: old mutation/index code could replace the object stored under an identity without the current explicit rejection of a live node/relationship kind conflict. Such replacement could leave the graph different from what the caller intended.

What changed: conflicting Insert or Update requests fail before modifying the existing object.

Evidence and limits: old evaluator, old in-memory replacement and the current cross-kind tests. The tests check both directions. This is about a conflicting live identity, not a ban on recreating a properly deleted object.

Stopping a query could lose the ability to wait for its real worker

Old problem: common query stop removed the worker handle before waiting. Cancelling that wait could discard the handle while the task still existed. On timeout it requested task abortion without then proving the task had ended, and could return success and mark the query stopped. That is not proof that processing, transactions or resource use finished.

What changed: tracked graph work remains owned while a caller stops waiting. Graceful stop waits for admitted processing and its required handoff. An explicit force request is different from a successful graceful stop; interrupted owners may require reconstruction.

Evidence and limits: old stop code, owned work, cancelled-worker tests, graceful-stop tests and native execution tests. These establish named supported paths. Arbitrary private plugin threads are not automatically controlled; the complete deadline policy is still CW-06.

A failed stop could prevent retry, while final shutdown hid stop failures

Old problem: instance stop cleared its running flag even after failure; a subsequent stop rejected the already-stopped instance. Final shutdown logged stop errors and continued toward a successful return. Callers could lack a truthful, retryable account of resources still in use.

What changed: current instance cleanup retains member failures and unfinished ownership for retry. A failed component is not silently removed or considered repaired just because a stop attempt was made.

Evidence and limits: old instance stop/shutdown and current instance cleanup tests, including failed stop and cancelled shutdown. This does **not** close every cleanup problem: management store close/open cases remain in CW-08.

Why these fixes matter to transactions

The old transaction support was real: participating query state, input progress and output could share a commit. But an atomic commit cannot correct a queue that selected the wrong input, a recovery path that deletes valid history, or a stop path that claims work finished without waiting for it.

The fixes above improve those surrounding behaviors. New transaction and delivery options below extend the set of things that can participate; they do not turn arbitrary external services into one distributed transaction.

Not every fix during the migration was a defect in ComponentGraph

Some work restored source recovery, result access or consumer behavior while moving execution to ComputationGraph. That is necessary preservation work, not evidence that the old runtime never supplied those capabilities.

Likewise, new graph reconciliation, multicast journals and native receipt protocols had their own bugs and follow-up fixes. Their repair should not be counted as repairing a feature that existed in the old runtime. Shared query edge cases still tracked in CW-12 and CW-13 are not claimed fixed by this comparison.

Things applications can now do

Here **new** means supplied directly by the framework, not something impossible for an application to have built itself before. Some entries generalize old query-specific behavior into reusable processing components.

Build a useful pipeline without inventing a query

Connect input to validation, normalization, enrichment and output steps, adding queries only where needed. Background services can run alongside them without pretending to produce data. The old built-in pipeline model centered on sources, queries and reactions; it did not supply this general managed processing model.

Give each input and output a checked data format

Components can have named inputs/outputs with explicit record formats. Incompatible connections and records that fail the declared checks are rejected. Different branches can attach their own context without modifying shared data. Old schema-discovery metadata was not the same as executable checks over arbitrary processing links.

Run reusable preprocessing once, with saved state and output

For example, normalize customer records before sending them to several queries. Supported durable middleware saves its state, input position and unfinished output together. Restart can resend saved output without running already committed preprocessing twice. Old middleware existed inside queries, not as this independently connected, recoverable stage.

Commit a sequence of stateful processing steps together

A supported transaction container runs opted-in steps in order and commits their state, input progress and final output together. It has no separate internal queues. Old atomic query evaluation already existed; the reusable ordered processing container is new. Ordinary arbitrary components, remote effects and nested transaction containers do not automatically participate.

Process a complete upstream transaction as one query input

A stock transfer may update two rows. Opted-in complete-source-transaction input lets one supported query process both under one transaction and publish final results rather than the intermediate half-transfer. The old input model supplied individual changes even when their original database transaction was larger. Explicit count, byte and duration limits apply; initial snapshot rows are not pretended to be one database transaction.

Wait a bounded time to combine streams by source time

The source-time merger can hold input within configured lateness, residence and size limits, then emit a nondecreasing source-time stream. Late input has an explicit fail, separate-route or discard policy. Its optional saved form survives restart. Old queues ranked available events; they did not supply this reusable waiting stage. It cannot prove that unseen older input will never arrive.

Choose delivery behavior separately for each connection

One connection can be a small volatile queue; another can retain saved history until handling completes. Selected connections also support full-message byte limits and bounded persistent read caches. Old specialized queues/logs/outboxes existed, but arbitrary processing links did not expose this common choice. Queue size is not an events-per-second limit or a total-process memory cap.

Save one message for several consumers and keep each consumer's obligation

For example, accounting and alerts can read one saved message history while acknowledging independently. A required consumer temporarily disconnecting does not release its outstanding work. Lossless mode keeps the message until every required consumer finishes. The old specialized fan-out was not this general saved multicast service with explicit membership and positions.

Share a native producer without merging its consumers' recovery

A source can feed multiple transformers/queries, and a transformer can feed multiple transformers and sinks. The graph reuses the producer instead of executing it once per branch. Native recovery now keeps an actual progress owner for every required consumer in a `ConsumerProgressSet`, including supported paths through stateless intermediates.

For example, a fast dashboard and a slower audit query can share one database change stream. They retain different snapshot boundaries and process at different speeds. Replay starts from retained physical source history, and each consumer filters changes already covered by its own checkpoint. The fast query cannot cause upstream feedback or pruning to pass the slow query's unfinished work. Unrelated logical and transport counters are not a combined database cursor.

Standalone transformers and Handled sinks can opt into graph-owned volatile `input_progress`; publication follows actual handling, output acceptance and continuations, including batched operation. That is not persistent state or an external-effect transaction. Atomic persistent consumers keep their real transaction-owned progress instead.

The native single-owner restriction was an incomplete migration, not an intended loss of old fan-out. This correction is distinct from QoS multicast: a consumer set describes actual processing owners, while a multicast journal retains messages and subscriber positions. Neither can fabricate the other's guarantee. See the owner/path validation, handled-input cases and mixed dataflow cases.

Membership is fixed for a live binding, but explicit reconciliation can enroll or remove members under each source's retention rules. It must stop the actual producer, drain admitted work and initialize the new owners before resuming. Pausing graph polling alone does not stop a source-owned connection from acknowledging upstream history. A disconnected reader is not a retired member.

Initialize one query from several native inputs

`CompositeBootstrapProvider` combines ordered, query-scoped providers through the existing bootstrap slot. A query joining PostgreSQL holdings with HTTP prices can receive both initial datasets instead of starting with a missing price table or running a second initialization engine.

The composite retains actual child handles and their cleanup/retirement leases. It streams rows, namespaces bounded child state and commits completion watermarks through the existing query transaction. Mixed preparation outcomes stop and rearm all children before reset/refresh retries. Cancellation or a partial snapshot cannot become a successful completed handover.

Host/Server expose `consumerProgress` for the owners of one producer and `compositeBootstrap` for the initializers of one query. Canonical definitions retain actual resource dependencies and secret configuration references. Neither recipe adds a graph executor or a shared transaction across databases. HTTP snapshots/live prices retain their existing non-durable replay limits. The composite implementation and real static/dynamic PG/HTTP cases define the supported scope.

Resolve a lost input reply without submitting the input twice

With durable input receipts enabled, retrying the same producer request and content can recover its saved acceptance result. A conflicting or expired retry fails instead of silently becoming fresh input. Source logs and saved source positions already existed; this reusable input-session/receipt protocol is new. It proves the selected input save, not completion of the entire application.

Resend committed output without adding it twice to the next saved queue

Supported output replay receipts recognize a previously accepted logical output while its receipt remains saved. Pending output also retains its required destinations through supported replacement. Old query outboxes already replayed output; the new service extends duplicate prevention to explicit native

processing-to-journal handoff. Receipt storage is bounded.

Include selected outgoing message journals in the processing commit

When the actual storage supports it, one processing owner and selected outgoing queues can save their changes in one transaction. This avoids a separate processing-to-queue save for that arrangement. It generalizes old query state/checkpoint/outbox commits; it is not a transaction over arbitrary queries or databases. The current shared group supports one processor and up to 256 active journals.

Record which destination operations really finished

A supported consumer can save the completed prefix of a result batch and resume the rest after failure. For a transactional local handler, one business change and its completion record can commit together. For a remote handler, stable keys can support destination-side duplicate prevention. Old consumer recovery existed; this is a reusable native per-operation completion service, not universal exactly-once remote delivery.

Check whether the whole selected path can provide the requested recovery

An application can request a recovery guarantee at a consumer and have the framework inspect upstream components, delivery choices and actual storage owners. An unknown capability or lossy connection cannot satisfy a stronger claim. Matching resource names do not prove shared transactions. The old specialized pipelines did not provide this check over arbitrary native paths. The check rejects inadequate setups; it does not create missing durability.

Save native pipeline definitions and make changes with revision checks

Supported definitions describe how to recreate components and resources. Configuration requests can be saved with retry receipts before construction; failed accepted components remain declared and inspectable. Unrelated commands can progress during selected slow updates. Old registration/configuration and graph metadata transactions existed, but not this general native reconstruction and managed-change system. Saving a definition is not proof it started.

State who owns a resource and when it may be released

Graphs distinguish application-owned resources from resources they must shut down themselves. Dependencies order creation and cleanup. Reported actual providers and native connections are inspectable; replacement checks can retain storage until its users finish. This generalizes old provider references. Private plugin resources still need a real ownership/reporting mechanism.

Explicitly clean up saved configuration records and rotate encryption keys

The redb configuration store supports bounded retry-receipt housekeeping, revision-checked deletion of named snapshots and atomic rotation of live records to a new key. These operate on the new saved-definition service, not query processing history. Deleting a configuration snapshot does not rewind results; rotating live records does not re-encrypt old backups or securely erase old pages.

Send a neighboring component control information without using its data queue

Authorized components can exchange readiness and application signals separately from data capacity. A handle for an old connection cannot control its replacement. Old lifecycle/status events existed; this general native neighbor-control service is new. Handler failure is recorded without permanently closing control.

Use native processing plugins with tracked execution and failures

Native plugins can provide sources, transformation steps, sinks and background services. The framework tracks supported work, keeps failed nodes visible and prevents reuse of an object damaged by a catchable panic. Independent native steps can run on different CPU workers. Old plugins and multicore query workers already existed; native roles and their graph-owned supervision are the change. The native interface is ABI 1.0.0/wire 2; the older SDK remains independently versioned at 0.17.0 while migration proceeds.

Shared progress and a retained source-position comparator use the separate `drasi_computation_plugin_consumer_progress_v1` extension, without changing the base ABI, wire version or existing recovery interface. The Host binds actual owners; plugins receive bounded read-only capabilities, not graph or storage ownership. The volatile `input_progress` tracker remains Host/graph-owned and does not cross the plugin boundary.

Put a risky plugin in another normal DrasiLib process

The new host can own normal child/grandchild DrasiLib processes and connect them using explicit native input/output links. The parent need not load the child's plugin code. Separate data and control paths support bounded queues, saved receipts and local or mutual-TLS links. Previously an application could arrange processes itself, but the framework did not supply this ownership/link system. Current recorded evidence is macOS/Linux, not Windows or a security sandbox.

What a transaction or acknowledgement actually promises

For a supported atomic query, index changes, saved input progress, result identity, current result rows and outgoing saved results commit together. A failure after that commit can resend saved output without evaluating the input again.

That does not mean the destination finished. There are four separate questions:

1. Did Drasi receive the input?
2. Was the input durably saved?
3. Did processing commit its state and output?
4. Did each destination actually complete its work?

A queue acknowledgement, HTTP 202 or input receipt answers only its documented question. Arbitrary fan-out can partially succeed. Several queries can publish at different times. An external effect may succeed before its local completion record is saved, so the destination may need a stable duplicate-prevention key. Supported transactional consumers narrow that risk only for participants in their actual transaction.

PostgreSQL delivery and exact-batch HTTP completion reference services exist. They do not automatically upgrade every HTTP/gRPC sink. Likewise, native Rust PostgreSQL/MySQL/SQLite source paths provide selected complete-transaction and snapshot behavior, but do not imply all are packaged as production native dynamic plugins. SQLite capture observes writes through its owned native handle, not arbitrary external writers.

Recovery must work across the actual source, processor, links, storage and destination. A persistent index alone cannot recover input a source can no longer replay. Missing history must fail or follow an explicitly selected reset/skip policy, not silently look like successful catch-up.

Changing, stopping and rebuilding a pipeline

Saving an accepted configuration means “this is now the requested definition,” not “every component is running.” Missing factories, unavailable secrets, occupied ports or failed startup remain visible. A client can resolve an uncertain configuration reply using the same request identity.

The current graph can preview revision-checked changes, replace components and connections, and manage supported resource changes. It checks internal commit prerequisites before changing live membership. External stop/cleanup effects cannot be undone as if deployment were one database transaction.

Normal stop drains supported admitted work. A timeout is not automatic force. Explicit force can interrupt an eligible complete registered resource-sharing group and require fresh runtime objects afterward. Cleanup does not repair a panicked object or erase pending outputs.

Saved definitions need real reconstruction instructions. An opaque Rust object is not automatically rebuildable. Restoring a configuration snapshot does not restore old processing data or undo external effects. Supported retirement of storage checks stopped users and pending work; an explicit loss permission is not permission to delete data, mark it handled or reuse an unsafe owner.

Separate time limits for each startup/cleanup step, each component or resource owner, and the whole shutdown are still unfinished (CW-06). Management-store close/open problems remain (CW-08). These are current implementation gaps, not just a statement that more tests would be desirable.

What Server and operators can use today

Server can read native graph state/configuration, add/remove/start/stop components, accept saved desired definitions, return receipts and reconcile them. These operations are under `/api/v1/instances/{instanceId}/computation`. Supported native resources include query storage, result catalogues, saved queues and native recovery services. The encrypted configuration store is opt-in; saved accepted state takes precedence over an obsolete startup seed.

Snapshots, saved query results and live subscriptions have implementations. Live access needs an actual connected result output and catalogue. Merely creating a native processing query does not make that wiring appear in every assembly. Completing standard reconstruction/wiring is CW-43. Named configuration snapshots are available through Rust management APIs; Server does not expose every low-level operation.

The full native-only Server/plugin/UI migration is **not complete** (CW-44 through CW-47). Native network plugins still lack some required templates, batching and snapshot/recovery replacement behavior. Native gRPC currently accepts only a batch size of one. Explicitly rejecting an old supported option is safer than silently ignoring it, but is still a missing replacement feature.

Inspection shows components, connections, reported resources and failures. Public topology data excludes secrets and detailed failure messages. Privileged configuration exports can contain secrets; a `no-store` HTTP header is not access control. Logs and profiling help diagnosis, but profile time includes waiting and is not pure CPU time.

What has been demonstrated, and what has not

Implemented and tested on specified paths

Recorded runs include exact query/transaction outcomes, saved queues and receipts, cancelled cleanup, failed replacement, native-library failures and actual process death with storage reopen. Protected test names and their specific checks are listed in the test inventory.

Published native fan-out checkpoint, 20 October 2026. The current Core/Server commits above include the following recorded qualification. These are named supported paths, not a claim that every connector or entire release matrix was rerun during this documentation refresh.

Area	Recorded scope
Generic graph and native Host	Source/transformer/query/sink fan-out, actual owners, per-consumer replay, normal/batched handled progress, provenance, cancellation and stopped-producer replacement; both Tokio flavors and rebuilt native libraries
PostgreSQL shared rows	Independent snapshots and counters, slow-member feedback, queued replay, memory/persistent owners and explicit late enrollment with one configured live slot
PostgreSQL shared whole transactions	Complete insert/update/delete effects without row prefixes, unequal snapshots, replay, draining two-to-three-owner replacement, persistent interruption and explicit recovery; 53 native transaction integration cases in the final family run
MySQL and SQLite native Rust sources	Actual persistent owner sets, coordinated snapshots and independent replay; MySQL snapshot/commit clocks and history-reader isolation; SQLite oldest-member pruning, bounded enrollment and safe removal
Composite bootstrap	Core and Host/Server recipe cases plus real PG/HTTP tests on both Tokio flavors, each using static and rebuilt dynamic providers; held snapshot/feedback fence, concurrent writes, live changes, rollback, restart and retirement hold/resume
Real Server consumers	Seven reproducible PostgreSQL configurations, actual Trading app provisioning, safe Getting Started setup, and fresh native GPU input/recovery proof

The PostgreSQL factories distinguish `drasi.postgres/shared-changes` from `drasi.postgres/shared-transactions`, with matching shared row/transaction bootstrap factories. Row mode supports ordinary memory queries without claiming process-restart durability; whole-transaction mode requires every query to be atomic, persistent and transaction-enabled. Per-query snapshots use temporary export/import resources without resetting, recreating or claiming ownership of the configured live slot. Old single-owner and row/transaction position formats are not reinterpreted as one another.

Trading's actual setup installs its complete 11-query fixed-member definition in one existing `/computation/components` batch, then creates its SSE reaction. It uses two shared native PostgreSQL sources and one native HTTP source, with the original 50-price seed served by the app's HTTP API and composed into query initialization. Recorded proof covers joined snapshots, actual API writes, live price fan-out/aggregates, rollback exclusion, SSE output and repeat setup without a graph revision change. Partial/conflicting bundles fail explicitly.

The YAMLs and Trading JSON are generated from real factory metadata and the app's actual query declarations, preserving secret `ConfigurationValue` references. Native Trading requires the matching local library set; it has no legacy/registry fallback. The example wrappers preserve existing data and slots and reject unsafe partial state. See the native source contracts, PG bootstrap contracts, Server recipes and qualifiers and Trading application.

Earlier checkpoints follow for historical context. Their counts and performance figures are not totals for the latest implementation.

The 16 October host-consolidation qualification passed 1,157 library tests in each middleware-enabled and no-default-features profile, 46 selected public query/recovery integration tests, 21 Host lifecycle/pipeline/wire tests, and 46 Server native integration tests. Existing ignored cases were retained. The public integration run includes all four ordinary stop APIs on both Tokio runtime flavors and exact output from an unaffected sibling query. Core and Server lint, formatting, generated architecture and protected-test inventory checks also passed. This was not a rerun of the entire connector test matrix. The durable network source-stop issue was not fixed by that host-consolidation checkpoint. A separate 16 October follow-up corrected the test's use of graceful stop where it required forced interruption, and fixed a real retry defect: after a graceful deadline, the next stop must reissue the withdrawn pause. Separate HTTP/gRPC tests preserve exact commit/reopen/delivery outcomes for graceful completion, forced interruption and timeout/retry. This is a ComputationGraph lifecycle correction, not another old ComponentGraph defect.

The same follow-up removes GPU Lab's pre-native unsupported capability marker. Its native PostgreSQL transaction source, single atomic configuration query, matching policy fingerprints and stale-worker rejection implement the policy-context consistency contract. Full acceptance now checks real native input/recovery before writer/restart/scenario behavior and the capability declaration. Two consecutive complete acceptance runs passed on matching current-source images through Podman. Restart verification now checks readiness on the exact captured rows, rather than checking one observation and comparing a later, potentially unconverged snapshot. All saved-plan, configuration, identity and command-receipt assertions remain. This is not a claim of one atomic transaction across plan commits, simulated GPU execution and browser updates.

The following throughput figures belong to the original reference measurement, not a new before/after benchmark of the host-consolidation change. The local independent-execution reference used eight CPU-heavy native queries, 5,000 inputs and three repetitions. It checked 480,000 results, including single-thread correctness. Median two/four-worker throughput was 1.926/3.478 times the one-worker result, with measured CPU overlap. This meets the selected reference targets; it is not a universal scaling or production-latency promise. Workload tools and an isolated resource soak provide additional bounded evidence, not a many-day leak guarantee.

Normal child/grandchild hosting has real-process macOS/Linux tests. Selected query-reset and merger crashes reopen real RocksDB storage. A prepared macOS/Linux-VM mutual-TLS test checks lost/delayed replies, version/authentication rejection, exact outputs and persistent reopen. That is useful network

evidence, not qualification of every physical-machine failure or production partition.

Existing code with incomplete wider evidence

The original 48-row test ledger reported 20 covered, 23 partial and five unqualified named scenarios. Those are historical scenario counts, not current feature-completion totals. Later shared-source, composed-bootstrap and real Server evidence is identified above; it does not automatically reclassify every older ledger row. Wider qualification still includes production-duration outages, broader clock/failure combinations and plugin/platform lifetime behavior beyond the named cases.

An earlier 28-profile compatibility run and the later selected execution tests are different checkpoints. Neither should be described as a freshly repeated full current-binary connector matrix. Selected GPU/browser/restart checks alone also do not replace the full native-input and policy-context acceptance gate.

Known unfinished implementation

The complete startup/shutdown deadline policy is CW-06. Local status updates still rebuild whole-graph mappings, and acceptable graph-size cost is not demonstrated (CW-31). The completed native fan-out and consumer work does not close the broader CW-43 through CW-47 migration or cleanup stage 6; stages 7 through 9 remain pending in the cleanup plan.

Specific remaining boundaries include native database schema discovery, MySQL/SQLite dynamic **source** packaging and Server registration, and making query execution an optional native plugin rather than a built-in processor. Shared native-source recovery has qualified drained reconstruction and drain-barrier member replacement, not cold reconstruction with undrained persistent input-pipe frames. Handled owner progress is not evidence of all queued transport allocations. HTTP snapshots/live prices have no added durable or atomic replay guarantee. Other inventory items distinguish confirmed gaps from findings not yet reverified.

Deliberate limits, not hidden promises

In-process scheduling is cooperative. A synchronous callback that never yields, started blocking work, a native abort or a memory fault cannot all be safely interrupted by a timeout. Separate processes can contain a child's crash, but current hosting is not a security sandbox that prevents file or network access.

Links currently accept the three standard graph-change, query-change and complete-source-transaction formats, not arbitrary executable validators sent by a peer. Transactions and shared Rust resources stay inside one instance. There is no automatic distributed transaction, remote machine provisioning, state migration, graph partitioning or universal exactly-once external effect.

Byte limits bound specified queues/caches, not all process memory. Delivery history and retry protection depend on their retention rules. Saved configuration receipts are indefinite by default unless housekeeping is selected. A slow required consumer can hold back a producer. Lossy delivery, reset and skip policies must be selected explicitly rather than inferred from successful startup.

Code and further reading

- Cleanup plan: current migration stages and scoped completion.

- Review inventory: broader findings, evidence and implementation/testing complexity, including historical checkpoints.
- Developer guide and maintainer's guide: using and maintaining the current code.
- Transactions, delivery/recovery, middleware and source-time merging: the supported choices.
- Managed configuration, native plugins, network plugins, child host and instance links.
- Recorded execution/isolation plan: approved requirements and selected evidence. Its earlier blanket completion statement is superseded by the current work inventory, not silently treated as proof.